



Edge-based quantum approximate optimization algorithm for MAX-CUT problem

Youngjin Seo¹ · Jun Heo¹

Received: 31 July 2024 / Accepted: 25 August 2025 / Published online: 23 September 2025
© The Author(s) 2025

Abstract

Quantum computing has emerged as a promising paradigm to tackle computationally intensive problems that classical computers struggle with. In this study, we explore the application of the Edge-based quantum approximate optimization algorithm (QAOA) to the MAX-CUT problem, a well-known combinatorial optimization challenge. MAX-CUT aims to partition the vertices of a graph into two subsets such that the number of edges between the subsets is maximized. We define the edge-based MAX-CUT problem and propose a method for applying QAOA specifically tailored to this formulation. We conduct simulations using IBM's Qiskit framework, examining both vertex-based and edge-based QAOA implementations across various graph structures. Our results highlight the comparative performance of these approaches in terms of solution quality and computational efficiency. Specifically, we analyze the impact of different graph sizes and edge densities on the complexity and CNOT gate counts of the proposed algorithms. This analysis provides insights into leveraging quantum computing for combinatorial optimization tasks, particularly focusing on the implications for practical applications and future research directions.

Keywords MAX-CUT problem · Quantum algorithm · Quantum approximate optimization algorithm · Noisy intermediate-scale quantum

1 Introduction

Research on quantum algorithms operating on noisy intermediate-scale quantum (NISQ) devices is actively being conducted. Quantum computers, capable of computing with small numbers of qubits and short circuit depths, are known as NISQ computers [1]. The variational quantum algorithm (VQA) is one of the representative algorithms used in the NISQ era [2–10]. Among the algorithms within the VQA,

✉ Jun Heo
junheo@korea.ac.kr

Youngjin Seo
cherishiz@korea.ac.kr

¹ School of Electrical Engineering, Korea University, Seoul 02841, Republic of Korea

the variational quantum eigensolver (VQE) is noted for estimating the ground-state eigenvalues and eigenvectors of a given Hamiltonian [11, 12]. Similarly, the quantum approximate optimization algorithm (QAOA) is known for approximately solving combinatorial problems [5].

Introduced in 2014 by Farhi, Goldstone, and Gutmann, the quantum approximate optimization algorithm (QAOA) is a hybrid quantum classical algorithm that leverages the strengths of both quantum and classical computing paradigms [5]. It employs a parameterized quantum circuit to effectively address combinatorial optimization problems, such as the MAX-CUT problem, by preparing an initial quantum state, applying the problem Hamiltonian, executing the mixing (driver) Hamiltonian, and concluding with a measurement. To achieve high performance, the algorithm iterates the problem Hamiltonian and mixing Hamiltonian multiple times, where the number of iterations is denoted by p .

Various studies have explored the application of QAOA to the MAX-CUT problem. In [13], the exact formula for QAOA's performance at $p = 1$ is presented. Zhou et al. conducted an analysis of QAOA's effectiveness, its non-adiabatic mechanism, and practical implementations in MAX-CUT problems [14]. Bravyi et al. proposed the Recursive QAOA (RQAOA), showcasing its superior performance compared to the Goemans–Williamson (GW) algorithm [15] in specific scenarios [16]. Bae and Lee provided analytical evidence demonstrating RQAOA's competitive edge over QAOA for the MAX-CUT problem on complete graphs, particularly in terms of approximation ratio [17]. Herrman et al. introduced the multi-angle QAOA (MA-QAOA), optimizing angle parameters for each edge and node to achieve a higher approximation ratio than QAOA [18].

Moreover, there have been various studies that evaluate the performance of QAOA for the MAX-CUT problem across different graph models and structures, conducting detailed simulations to analyze its effectiveness [19–21]. Bravo-Prieto et al. introduced the variational quantum linear solver (VQLS), a hybrid quantum classical algorithm tailored for solving linear systems on near-term quantum computers [4]. Furthermore, several research efforts have leveraged the Lechner–Hauke–Zoller (LHZ) architecture [22] to enhance QAOA performance by transforming problem encodings. These approaches have demonstrated significant improvements in specific use cases [23–25]. Additionally, studies have investigated the feasibility of QAOA on NISQ hardware, demonstrating its applicability to large-scale combinatorial optimization problems. These works explored QAOA's performance on higher-order Ising models and validated its functionality on devices such as IBM quantum processors [26, 27].

When solving the MAX-CUT problem with QAOA, the algorithm operates by assigning vertex information to qubits. This paper proposes a method to apply QAOA by considering the transition from the conventional vertex-based MAX-CUT problem to an edge-based formulation. In addition to presenting the edge-based approach, we propose an initial assignment and a dynamic edge reassignment method to further enhance the efficiency of the algorithm.

The remainder of this paper is organized as follows. Section 2 reviews essential QAOA concepts and circuit design principles, along with relevant performance metrics. Section 3 presents the proposed edge-based algorithm in detail. First, Sect. 3.1 defines the edge-based MAX-CUT problem, adapting definitions from [28]. Next,

Sect. 3.2 outlines QAOA techniques tailored for edge-based formulations, while Sect. 3.3 describes different edge assignment methods and a reassignment process that mitigates gate overhead. Simulation results, shown in Sect. 4, evaluate the performance of the proposed approach in two aspects. First, approximation ratios were compared up to level 3 across 10 different graph instances. Second, circuit depths and CNOT gate counts were analyzed over 1000 random graphs of fixed size to assess the structural efficiency of each method. These results collectively demonstrate the efficacy of our approach.

2 Previous work

2.1 QAOA for a MAX-CUT problem

The QAOA is an algorithm used to solve combinatorial optimization problems, with the MAX-CUT problem being a prominent example. By harnessing quantum resources, it aims to approximate solutions for such optimization challenges. Here, we introduce the QAOA method aimed at achieving an optimized solution for the MAX-CUT problem [5].

Given the graph $G = (V, E)$, let V be the set of n vertices and E be the set of edges with $|E| = m$. Then $(j, k) \in E$ where j and k are elements of V . The cost function for the MAX-CUT problem is defined as follows:

$$C(z) = \frac{1}{2} \sum_{(j,k) \in E} (1 - z_j z_k), \quad (1)$$

where $z = (z_1, z_2, \dots, z_n)$, and $z_i \in \{1, -1\}$. Here, z_i represents the value assigned to vertex i . For each of the m edges $(j, k) \in E$, the cost function increases by 1 whenever $z_j \neq z_k$ (i.e., $z_j z_k = -1$), indicating a cut edge. Hence, solving this problem amounts to maximizing $C(z)$ by maximizing the number of cut edges.

For the MAX-CUT cost function, QAOA defines the following problem Hamiltonian H_P and mixing (driver) Hamiltonian H_M :

$$H_P = \sum_{(j,k) \in E} \frac{1}{2} (I - Z_j Z_k), \quad (2)$$

$$H_M = \sum_{i=1}^n X_i, \quad (3)$$

where I represents the identity operator, and Z and X denote the Pauli σ^z and σ^x operators, respectively. Using H_P and H_M , the p -level unitary operator $U_p(\beta, \gamma)$ is defined as:

$$U_p(\beta, \gamma) = e^{-i\beta_p H_M} e^{-i\gamma_p H_P} \dots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_P}, \quad (4)$$

where $\beta = (\beta_1, \beta_2, \dots, \beta_p)$ and $\gamma = (\gamma_1, \gamma_2, \dots, \gamma_p)$. Here, β_i and γ_i are the angle parameters at the i -level.

In QAOA, the p -level unitary operator $U_p(\beta, \gamma)$ is used to obtain the final quantum state $|\psi_p(\beta, \gamma)\rangle$. This final quantum state is derived by applying the p -level unitary operator to the initial state $|+\rangle^{\otimes n}$.

$$|\psi_p(\beta, \gamma)\rangle = U_p(\beta, \gamma) |+\rangle^{\otimes n} = e^{-i\beta_p H_M} e^{-i\gamma_p H_P} \dots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_P} |+\rangle^{\otimes n}. \quad (5)$$

By measuring the final quantum state, the expectation value of the cost function can be determined. The expectation value $F_p(\beta, \gamma)$ of the p -level QAOA for the angle parameters β and γ is defined as follows:

$$F_p(\beta, \gamma) = \langle \psi_p(\beta, \gamma) | H_P | \psi_p(\beta, \gamma) \rangle. \quad (6)$$

The final step is to optimize the angle parameters β and γ using a classical optimizer, which may require multiple evaluations of $F_p(\beta, \gamma)$ on a quantum device. Through this procedure, the optimal parameters are obtained as $(\beta^*, \gamma^*) = \arg \max_{\beta, \gamma} F_p(\beta, \gamma)$.

Using the optimal angle parameters (β^*, γ^*) , the approximation ratio R is obtained as follows:

$$R = \frac{F_p(\beta^*, \gamma^*)}{C_{\max}}, \quad (7)$$

where $C_{\max} = \max_z C(z)$. This ratio measures how close the QAOA solution is to the maximum possible cost, $C_{\max}$, and thus serves as a key performance metric. The overall QAOA procedure for the MAX-CUT problem is summarized in Algorithm 1.

Algorithm 1 Quantum Approximate Optimization Algorithm for the Vertex-Based MAX-CUT Problem

- 1: Prepare n qubits in the initial state $|+\rangle^{\otimes n}$, where each qubit is in the superposition state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.
(Each qubit is assigned to a vertex in the graph G .)
 - 2: Perform R_{ZZ} gate operations with angle γ on the qubits corresponding to the connected vertices in graph G .
 - 3: Once rotation gate operations are performed for all edges, perform the mixing Hamiltonian operation with angle β on all qubits.
 - 4: Repeat steps 2 to 3 for p levels, where γ and β can vary at each level.
 - 5: Measure the final quantum state and calculate the cost based on the obtained results.
 - 6: Optimize the angles to maximize the calculated cost.
-

3 Proposed method

The method for solving the MAX-CUT problem using QAOA, as described in Sect. 2, involves the operation of the problem Hamiltonian. This operation is implemented in quantum circuits using R_{ZZ} gates. Specifically, the QAOA design strategy for the MAX-CUT problem involves computing R_{ZZ} gates for each edge in the graph. R_{ZZ} gate operations can be decomposed into two CNOT gates and one single-qubit rotation gate. This decomposition results in significant computational costs and leads to a high circuit depth. In the NISQ era, it is crucial to employ quantum algorithms with lower

complexity. Therefore, this paper proposes methods to reduce the number of gate operations in QAOA for solving the MAX-CUT problem. The proposed approach focuses on designing QAOA based on the edges of the graph.

3.1 Edge-based MAX-CUT problem

The MAX-CUT problem defines its cost function based on the states of vertices, resolving the problem using vertex information. An edge between two vertices is considered cut if their states differ. The proposed approach aims to utilize edge states rather than vertex states in solving the MAX-CUT problem, requiring a definition of the MAX-CUT problem based on edges. In [28], edge states in the MAX-CUT problem are considered such that an edge is deemed cut if its state is 1, and not cut if its state is 0. This definition is used to formulate the edge-based MAX-CUT problem.

Assuming the graph cannot be partitioned into two or more disconnected subgraphs, given a graph G with n vertices and m edges, each vertex in G can have two possible states, resulting in 2^n potential assignments of vertex information. However, from the perspective of edge cuts, there are $2^{(n-1)}$ distinct scenarios. This is because the outcomes of edge cuts remain identical regardless of the specific state (0 or 1) of an arbitrary vertex. Consequently, when transitioning from a vertex-based MAX-CUT problem to an edge-based MAX-CUT problem, only $n - 1$ bits of information are required.

In the edge-based MAX-CUT problem, information is assigned to edges. This information, which determines the cut state of the corresponding edge, is represented as binary data. Typically, from the m edges (where $m \geq n - 1$), we need to select $n - 1$ edges to assign this information. The selected edges must form a set that includes all vertices in the graph. For instance, consider the complete graph with four vertices shown in Fig. 1a. In this case, the chosen edge set must include all vertices numbered from 1 to 4.

In Fig. 1b, c, the solid lines represent the edges in the selected edge set, while the dotted lines indicate other edges. Choosing edges such as (1, 2), (2, 4), and (3, 4) as shown in Fig. 1b is valid. However, selecting edges like (1, 4), (1, 2), and (2, 4) as shown in Fig. 1c is invalid because it does not provide information about vertex 3.

If a graph with n vertices has $n - 1$ edges and is fully connected, its structure exactly corresponds to a spanning tree graph [29]. In other words, selecting the edges for the edge-based MAX-CUT problem in graph G is equivalent to selecting a spanning tree, denoted as G_e , of G with n vertices. G_e can take various forms depending on the structure of G . In the case of Fig. 1a, the edge assignment shown in Fig. 1b can be regarded as a valid configuration of G_e .

The next step is to assign cut information to the selected edges. Based on this cut information, vertex information is then assigned in the original graph. The cut information for unselected edges is automatically determined once the vertex information is established. Using this, the cost function can be calculated.

For instance, consider the complete graph with four vertices shown in Fig. 1a. Suppose we select the edges (1, 2), (2, 4), and (3, 4). The information for these selected edges can be represented as a 3-bit string. If the string is “101,” where “1” indicates

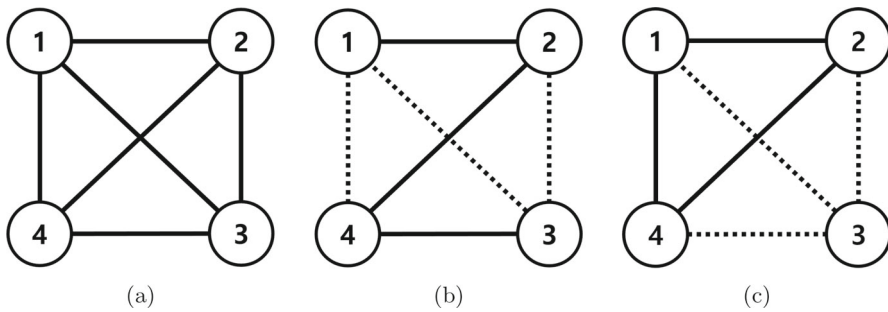


Fig. 1 **a** A complete graph with 4 vertices. **b, c** Examples of edge assignment for solving the edge-based MAX-CUT problem. Solid lines represent selected edges, and dotted lines represent unselected edges. Since there are 4 vertices, 3 edges need to be selected. In **b**, edges (1, 2), (2, 4), and (3, 4) are assigned, while in **c**, edges (1, 2), (2, 4), and (1, 4) are assigned. Among them, **b** illustrates a valid edge assignment, whereas **c** represents an invalid one

that an edge is cut and “0” indicates that it is not cut, then edges (1, 2) and (3, 4) are cut, while edge (2, 4) is not. Consequently, since edge (1, 2) is cut, vertices 1 and 2 must be in different states. Similarly, vertices 3 and 4 must also be in different states because edge (3, 4) is cut. Since edge (2, 4) is not cut, vertices 2 and 4 must be in the same state. This results in vertex states of either “0101” or “1010.” Therefore, in the edge-based MAX-CUT problem, we select $n - 1$ edges that include all vertices. Assigning “0” to an edge means it is not cut, while assigning “1” means it is cut. This allows us to allocate information to edges and then convert it to vertex states.

We define the functions $h_{0,j}$ and $h_{1,j}$ to convert from the edge-based state x to the vertex-based state y while specifying the state at a particular index j . Specifically, $h_{0,n}$ ensures that the state at the n th index is 0, and $h_{1,n}$ ensures that the state at the n th index is 1. For example, consider an edge-based state x represented by the binary sequence “101” in Fig. 1b. Applying $h_{0,1}$ to this sequence would yield a vertex-based state where the first index is set to 0, resulting in “0101.” On the other hand, applying $h_{1,1}$ would result in the first index being set to 1, giving “1010.” This approach ensures that the conversion process is unambiguous by specifying the state at a particular index.

In the context of the edge-based MAX-CUT problem, the process involves selecting $n - 1$ edges to assign cut information and determining vertex states based on this information. From the perspective of the vertex-based MAX-CUT problem, this process may appear unnecessary. However, it is essential for the operation of edge-based QAOA, enabling the construction of quantum circuits based on the information assigned to edges and yielding advantages in computational complexity.

3.2 QAOA for edge-based MAX-CUT problem

In this section, we introduce the method of applying QAOA to the edge-based MAX-CUT problem defined in Sect. 3.1.

Since the edge-based MAX-CUT problem assigns cut information to edges, the form of the problem Hamiltonian operation in QAOA differs from that in vertex-based QAOA. For a graph G with n vertices and m edges, the problem Hamiltonian

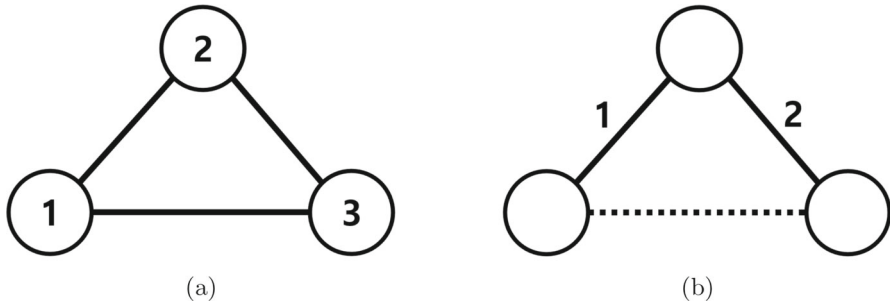


Fig. 2 Example of a triangular graph. **a** Triangular graph used for solving the vertex-based MAX-CUT problem with QAOA. The three vertices are assigned to three qubits, and the numbering on the vertices indicates the qubit indices. **b** Triangular graph used for solving the edge-based MAX-CUT problem with QAOA. Solid lines represent edges assigned to qubits, indexed as 1 and 2, while the dashed line represents an edge not assigned to a qubit

$H_{P'}$ of the edge-based QAOA is defined as follows:

$$H_{P'} = \sum_{(j,k) \in E} \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{jk}} Z_{f_q(j,k)} \right), \quad (8)$$

where l_{jk} denotes the length of the path between vertices j and k in the spanning tree G_e . This path is determined solely by the structure of G_e , and its maximum length does not exceed $n - 1$. The function $f_q(j, k)$ denotes the qubit index of the edge assigned at the q th step along the path from vertex j to vertex k .

The final quantum state of the edge-based QAOA is expressed as:

$$\left| \psi_p'(\beta, \gamma) \right\rangle = e^{-i\beta_p H_{M'}} e^{-i\gamma_p H_{P'}} \dots e^{-i\beta_1 H_{M'}} e^{-i\gamma_1 H_{P'}} |+\rangle^{\otimes(n-1)}, \quad (9)$$

where $H_{M'} = \sum_{i=1}^{n-1} X_i$ represents the mixing Hamiltonian of the edge-based QAOA.

The measurement of the final quantum state in Eq. (9) must be converted into corresponding vertex values for the vertex-based MAX-CUT problem. This conversion can be performed using the $h_{0,j}$ or $h_{1,j}$ functions described in Sect. 3.1. The optimization process is performed identically to that of vertex-based QAOA, and the final results are converted into vertex-based states to calculate the approximation ratio of the edge-based QAOA.

To provide a more detailed explanation of l_{jk} and $f_q(j, k)$, let us consider a triangular graph, as illustrated in Fig. 2a. In the vertex-based MAX-CUT problem, the three vertices can collectively have 8 possible states, ranging from “000” to “111.” From the perspective of edge cuts, these states can correspond to either “0” edges being cut or “2” edges being cut. This can be summarized in Table 1.

As shown in Table 1, the quantum state “000” in vertex-based QAOA is equivalent to “111” in terms of edge cuts. Similarly, “001” and “110” are equivalent, and other quantum states also have equivalent counterparts. Therefore, when considering a triangular graph in vertex-based QAOA, the information required to represent all

Table 1 Comparison of vertex-based and edge-based quantum states for a triangular graph in Fig. 2

Vertex-based quantum state	Number of cut edges (vertex-based)	Edge-based quantum state	Number of cut edges (edge-based)
$ 000\rangle$	0	$ 00\rangle$	0
$ 111\rangle$	0		
$ 001\rangle$	2	$ 01\rangle$	2
$ 110\rangle$	2		
$ 010\rangle$	2	$ 11\rangle$	2
$ 101\rangle$	2		
$ 011\rangle$	2	$ 10\rangle$	2
$ 100\rangle$	2		

The table shows the possible quantum states of the vertices and the corresponding number of cut edges in the vertex-based MAX-CUT problem. It also illustrates how these states translate to the edge-based quantum states and the associated number of cut edges

possible states is 2 bits. Consequently, in edge-based QAOA, as illustrated in Fig. 2b, the necessary information is assigned to 2 qubits, each corresponding to an edge. In edge-based QAOA, $|00\rangle$ indicates that no edges in the triangle are cut, which is equivalent to $|000\rangle$ and $|111\rangle$ in vertex-based QAOA. The four cases of the triangle's cut states are illustrated in Fig. 3. Figure 3a depicts the case in which no edges are cut, and in vertex-based QAOA, it can be represented by $|000\rangle$ and $|111\rangle$. In edge-based QAOA, it is represented by $|00\rangle$, where the two qubits correspond to edges (1, 2) and (2, 3). Figure 3b–d illustrate the graph configurations in which exactly two edges are cut. For instance, Fig. 3b shows the case where edges (2, 3) and (1, 3) are cut, represented in vertex-based quantum states as $|001\rangle$ or $|110\rangle$, and in the edge-based quantum state as $|01\rangle$.

In Fig. 2, let the edge (1, 2) be assigned to qubit 1, and the edge (2, 3) to qubit 2. In this case, the assigned edges (1, 2) and (2, 3) have path lengths $l_{12} = 1$ and $l_{23} = 1$, while the unassigned edge (1, 3) corresponds to a path of length $l_{13} = 2$ via the two edges (1, 2) and (2, 3). Finally, the problem Hamiltonian of the edge-based QAOA can be summarized as follows:

$$\begin{aligned}
 H_{P'} &= \sum_{(j,k) \in E} \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{jk}} Z_{f_q(j,k)} \right) \\
 &= \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{12}} Z_{f_q(1,2)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{23}} Z_{f_q(2,3)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{13}} Z_{f_q(1,3)} \right) \\
 &= \frac{1}{2} (I - Z_{f_1(1,2)}) + \frac{1}{2} (I - Z_{f_1(2,3)}) + \frac{1}{2} (I - Z_{f_1(1,3)} Z_{f_2(1,3)}) \\
 &= \frac{1}{2} (I - Z_1) + \frac{1}{2} (I - Z_2) + \frac{1}{2} (I - Z_1 Z_2), \tag{10}
 \end{aligned}$$

where f is a function that maps an assigned edge to its qubit index, as formally defined in Eq. (8). For example, $f_1(1, 2)$ is the qubit index of the first edge in the path representation for the assigned edge $(1, 2)$. In other words, because edge $(1, 2)$ is assigned, the path length is 1, and the first edge is $(1, 2)$ itself, whose qubit index is 1. Similarly, $f_1(2, 3)$ corresponds to the qubit index of edge $(2, 3)$, which is 2. In a slightly more complex case, if the path for edge $(1, 3)$ is represented by $(1, 2)$ and $(2, 3)$, then $f_1(1, 3) = 1$ and $f_2(1, 3) = 2$.

To operate the edge-based QAOA corresponding to Fig. 2b, the first step involves preparing qubits 1 and 2 in the $|++\rangle$ state. Subsequently, the problem Hamiltonian H_P operation is applied. When the path length is 1, a single Z -rotation gate is executed, and when the path length is 2, a ZZ -rotation gate is executed. At this stage, the rotation angle is γ . The corresponding quantum circuit is illustrated in Fig. 4b.

In Fig. 4a, the problem Hamiltonian H_P operation is performed using R_{ZZ} gates, repeated for each edge. However, in the edge-based QAOA circuit, two CNOT gates and one single rotation gate are performed after the two initial single rotation gates. The single rotation gate applied between the two CNOT gates mirrors the structure of the R_{ZZ} gate. From a complexity perspective, assuming that single rotation gate operations and CNOT gate operations each count as one execution step, the edge-based QAOA requires five fewer steps than the vertex-based QAOA. Note that in the circuit diagram, the rotation angles are labeled simply as γ and β for clarity. However, based on the definition of the problem Hamiltonian $H_P = \sum \frac{1}{2}(I - Z_j Z_k)$, the actual angle applied in the R_{ZZ} gate corresponds to $-\gamma$. Similarly, the mixing Hamiltonian $H_M = \sum X_i$ results in an overall rotation of 2β . These coefficients are omitted in the figure for readability, but must be taken into account when interpreting the circuit behavior.

For the triangular graph example in Fig. 2, the quantum state of the 1-level vertex-based QAOA can be expressed as follows.

$$\begin{aligned}
 |\psi\rangle &= U(\beta, \gamma) H^{\otimes 3} |0\rangle \\
 &= \frac{e^{-i\beta \sum_{j=1}^3 X_j} e^{i\frac{\gamma}{2} \sum_{(u,v) \in E} Z_u Z_v}}{2\sqrt{2}} \sum_{x=0}^7 |x\rangle \\
 &= \frac{e^{-i\beta \sum_{j=1}^3 X_j}}{2\sqrt{2}} \left\{ |0\rangle + |7\rangle + e^{-i2\gamma} (|1\rangle + |2\rangle + |3\rangle + |4\rangle + |5\rangle + |6\rangle) \right\} \\
 &= \frac{e^{-i\beta \sum_{j=1}^3 X_j}}{2\sqrt{2}} \left[\{ |0\rangle + e^{-i2\gamma} (|1\rangle + |2\rangle + |3\rangle) \} |0\rangle + \{ |3\rangle + e^{-i2\gamma} (|0\rangle + |1\rangle + |2\rangle) \} |1\rangle \right] \\
 &= \frac{e^{-i\beta \sum_{j=1}^3 X_j}}{\sqrt{2}} (|c_3\rangle |0\rangle + |d_3\rangle |1\rangle) \\
 &= e^{-i\beta \sum_{j=1}^2 X_j} \left(|c_3\rangle \frac{e^{-i\beta X}}{\sqrt{2}} |0\rangle + |d_3\rangle \frac{e^{-i\beta X}}{\sqrt{2}} |1\rangle \right) \\
 &= e^{-i\beta \sum_{j=1}^2 X_j} \left\{ |c_3\rangle \frac{1}{\sqrt{2}} (\cos \beta |0\rangle - i \sin \beta |1\rangle) + |d_3\rangle \frac{1}{\sqrt{2}} (\cos \beta |1\rangle - i \sin \beta |0\rangle) \right\}
 \end{aligned}$$

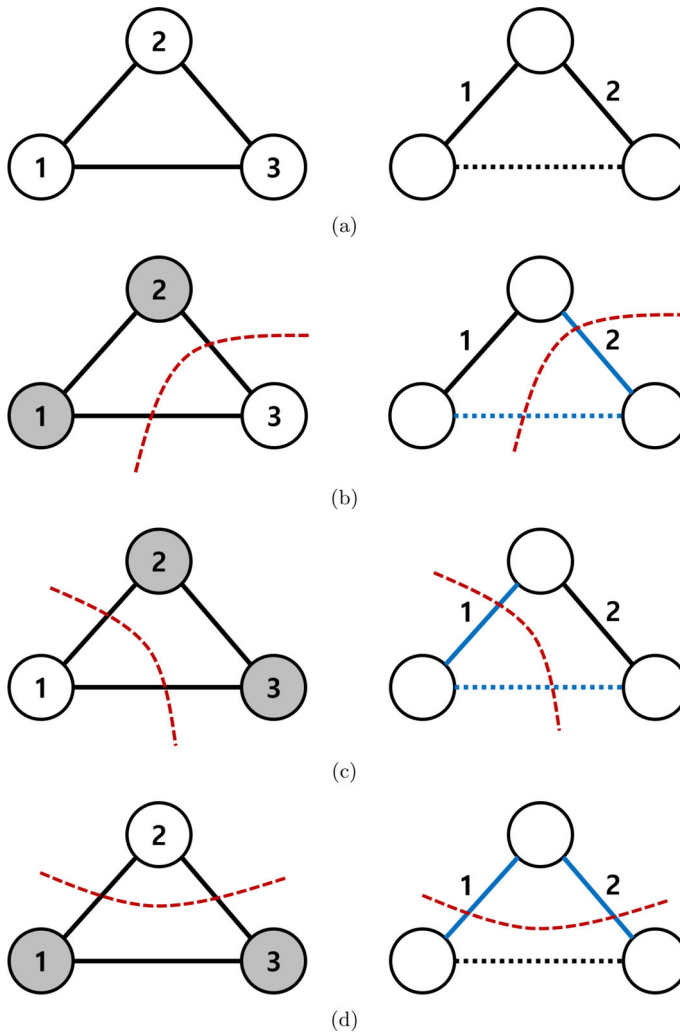


Fig. 3 Example of a triangular graph: the left side depicts the vertex-based graph, while the right side illustrates the edge-based graph. In the vertex-based approach, 3 qubits are allocated to vertices 1, 2, and 3. In contrast, the edge-based approach assigns 1 qubit per edge, resulting in 2 qubits being used for the two selected edges labeled as edges 1 and 2. Unassigned edges in the edge-based approach are indicated by dotted lines. Red dashed lines indicate cut edges. In the left column, vertices in the same state are distinguished using white and gray background colors, while in the right column, edges in the same state are represented by blue and black lines

$$\begin{aligned}
 &= e^{-i\beta \sum_{j=1}^2 X_j} \left\{ (\cos \beta |c_3\rangle - i \sin \beta |d_3\rangle) \frac{1}{\sqrt{2}} |0\rangle + (\cos \beta |d_3\rangle - i \sin \beta |c_3\rangle) \frac{1}{\sqrt{2}} |1\rangle \right\} \\
 &= (\cos \beta |a_3\rangle - i \sin \beta |b_3\rangle) \frac{1}{\sqrt{2}} |0\rangle + (\cos \beta |b_3\rangle - i \sin \beta |a_3\rangle) \frac{1}{\sqrt{2}} |1\rangle, \quad (11)
 \end{aligned}$$

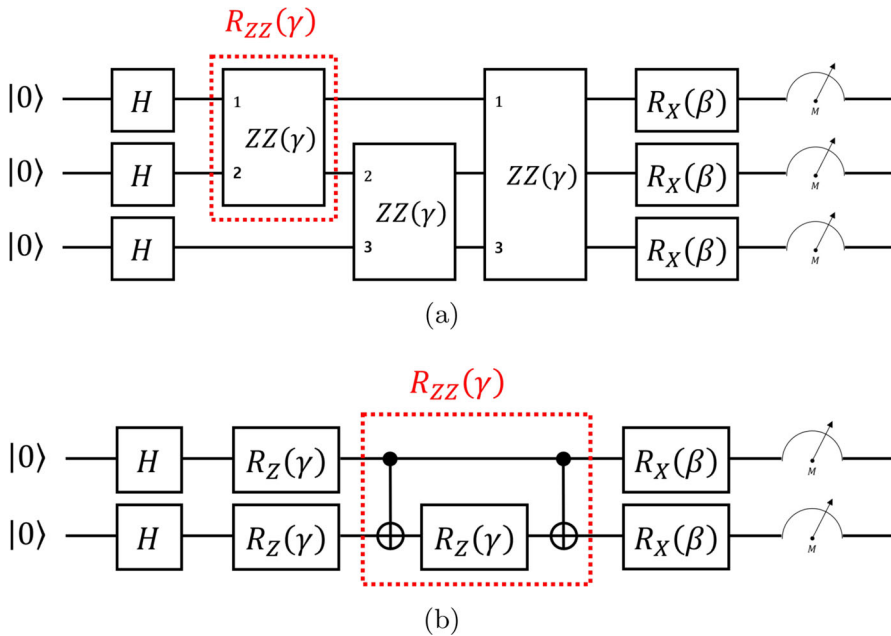


Fig. 4 Vertex-based and edge-based quantum circuits for the triangular graph example. **a** Vertex-based quantum circuit. H denotes the Hadamard gate. The red dotted box indicates the rotation operator $R_{ZZ}(\gamma)$ applied to qubits 1 and 2. $R_X(\beta)$ denotes the rotation X gate with angle β . **b** Edge-based quantum circuit. $R_Z(\gamma)$ denotes the rotation Z gate with angle γ

where $|a_3\rangle$, $|b_3\rangle$, $|c_3\rangle$, and $|d_3\rangle$ represent 2-qubit superposition states, and their specific forms are as follows. The states $|c_3\rangle$ and $|d_3\rangle$ represent quantum states after the application of the problem Hamiltonian, reflecting Z -axis rotations corresponding to the number of edge cuts. In this case, for $|c_3\rangle$, the states $|1\rangle$, $|2\rangle$, and $|3\rangle$ each correspond to 2 edge cuts in the triangular graph, resulting in a rotation by angle γ around the Z -axis, while state $|0\rangle$ has 0 edge cuts and undergoes no rotation.

$$|c_3\rangle = \frac{1}{2}\{|0\rangle + e^{-i2\gamma}(|1\rangle + |2\rangle + |3\rangle)\}, \quad |d_3\rangle = \frac{1}{2}\{|3\rangle + e^{-i2\gamma}(|0\rangle + |1\rangle + |2\rangle)\},$$

$$|a_3\rangle = e^{-i\beta \sum_{j=1}^2 X_j} |c_3\rangle, \quad |b_3\rangle = e^{-i\beta \sum_{j=1}^2 X_j} |d_3\rangle, \quad (12)$$

where $|c_3\rangle = X^{\otimes 2} |d_3\rangle$. Due to the relationship between $|c_3\rangle$ and $|d_3\rangle$, it follows that $|a_3\rangle = X^{\otimes 2} |b_3\rangle$ as well.

$$\begin{aligned} |a_3\rangle &= e^{-i\beta \sum_{j=1}^2 X_j} |c_3\rangle = X_1 X_2 X_1 X_2 (e^{-i\beta \sum_{j=1}^2 X_j} |c_3\rangle) \\ &= X_1 X_2 (e^{-i\beta \sum_{j=1}^2 X_j} X_1 X_2 |c_3\rangle) = X_1 X_2 (e^{-i\beta \sum_{j=1}^2 X_j} |d_3\rangle) \\ &= X_1 X_2 |b_3\rangle. \end{aligned} \quad (13)$$

Considering the case with n vertices, the 1-level quantum state of vertex-based QAOA can be expressed as follows:

$$\begin{aligned} |\phi\rangle &= U(\beta, \gamma) H^{\otimes n} |0\rangle \\ &= \frac{e^{-i\beta \sum_{j=1}^n X_j}}{\sqrt{2}} (|c_n\rangle |0\rangle + |d_n\rangle |1\rangle) \\ &= (\cos \beta |a_n\rangle - i \sin \beta |b_n\rangle) \frac{1}{\sqrt{2}} |0\rangle + (\cos \beta |b_n\rangle - i \sin \beta |a_n\rangle) \frac{1}{\sqrt{2}} |1\rangle. \quad (14) \end{aligned}$$

In Eq. (14), similar to the triangular case, $|a_n\rangle = X^{\otimes(n-1)} |b_n\rangle$. When measuring the state $|\phi\rangle$, the values a_n0 or b_n1 appear with probability $\frac{1}{2} \cos^2 \beta$, while a_n1 or b_n0 appear with probability $\frac{1}{2} \sin^2 \beta$. Here, a_n0 and b_n1 are inverted states that yield the same edge-cut outcome for the MAX-CUT problem. Thus, from the perspective of edge cuts, the measurement results in an outcome equivalent to a_n0 with probability $\cos^2 \beta$ or an outcome equivalent to a_n1 with probability $\sin^2 \beta$.

The 1-level quantum state of the edge-based QAOA for the triangular example is as follows:

$$\begin{aligned} |\psi'\rangle &= e^{-i\beta H_{M'}} e^{-i\gamma H_{P'}} H^{\otimes 2} |0\rangle \\ &= e^{-i\beta \sum_{j=1}^2 X_j} e^{i\gamma/2 \sum_{(j,k) \in E} (\otimes_{q=1}^{l_{jk}} Z_{fq(j,k)})} H^{\otimes 2} |0\rangle \\ &= e^{-i\beta \sum_{j=1}^2 X_j} e^{i\gamma/2 (Z_1 + Z_2 + Z_1 Z_2)} H^{\otimes 2} |0\rangle \\ &= \frac{e^{-i\beta \sum_{j=1}^2 X_j} e^{i\gamma/2 (Z_1 Z_2)}}{2} \left\{ |0\rangle + e^{i\gamma} (|1\rangle + |2\rangle) + e^{-i2\gamma} |3\rangle \right\} \\ &= \frac{e^{-i\beta \sum_{j=1}^2 X_j}}{2} \left\{ |0\rangle + e^{-i2\gamma} (|1\rangle + |2\rangle + |3\rangle) \right\} \\ &= e^{-i\beta \sum_{j=1}^2 X_j} |c_3\rangle \\ &= |a_3\rangle. \quad (15) \end{aligned}$$

Although the quantum state of the edge-based QAOA for the triangular graph appears similar to that of the vertex-based QAOA, they differ significantly for more complex graphs, since the edge-based QAOA assigns qubits to edges whereas the vertex-based QAOA assigns qubits to vertices. Nevertheless, both methods share a common characteristic: their angles are rotated based on the number of edge cuts. Hence, as mentioned in Sect. 3.1, the measurement results of edge-based QAOA and vertex-based QAOA can be compared using the function $h_{0,n}$.

Immediately after performing the problem Hamiltonian ($H_{P'}$) operation on a graph with n vertices, the quantum state of the edge-based QAOA becomes a superposition of 2^{n-1} states, each rotated by a phase according to the edge cuts. By applying $h_{0,n}$ to the bit string index of each superposed state, the resulting indices match those of the superposed state $|c_n0\rangle$ in the vertex-based QAOA. Thus, the measurement results in vertex-based QAOA correspond to those in the edge-based QAOA. Consequently, the

vertex-based QAOA yields the same measurement results as the edge-based QAOA when $\beta = 0$ degrees (i.e., no mixing Hamiltonian operation is applied).

Given a graph G with n vertices and m edges, the proposed method is summarized by Algorithm 2. In step 1 of Algorithm 2, G_e is the edge-assigned graph, which is a spanning tree graph with n vertices as mentioned in Sect. 3.1. The graph G_e can take various forms, and depending on G_e , the form of the edge-based QAOA problem Hamiltonian may vary.

Algorithm 2 Quantum Approximate Optimization Algorithm for Edge-Based MAX-CUT Problem

- 1: Set G_e as the edge-assigned graph, which is a spanning tree with n vertices.
 - 2: Prepare $n - 1$ qubits in the initial state $|+\rangle^{\otimes(n-1)}$, where each qubit is in $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.
(Each qubit corresponds to one of the $n - 1$ edges in G_e .)
 - 3: For all qubits, perform the problem Hamiltonian $H_{p'}$ and the mixing Hamiltonian $H_{M'}$ operations with angles γ and β .
 - 4: Repeat steps 2–3 for p levels, allowing different angle parameters γ and β in each iteration.
 - 5: Measure the final quantum state and compute the cost from the obtained measurement results.
 - 6: Optimize the angle parameters to maximize the computed cost.
-

A detailed example illustrating the algorithm's steps is provided in Appendix A. Performance comparison for simple examples and complexity/CNOT gate count results for larger graphs are presented in Sect. 4.

3.3 Initial assignment and dynamic edge reassignment

The process of assigning edges to qubits in step 1 of Algorithm 2 can significantly impact the overall computational complexity. Consider a simple example, depicted in Fig. 5a. Depending on the edge assignment method in step 1, the edges can be assigned as shown in either Fig. 5b or c. Both Fig. 5b, c represent valid edge assignments, as each includes all vertices in the edge set. When constructed as quantum circuits, they correspond to Fig. 5d, e respectively.

In Fig. 5d, the circuit is constructed using edges with path lengths of either 1 or 2. For edges with a path length of 1, no CNOT gate operations are required. In contrast, for edges with a path length of 2, an R_{ZZ} gate is applied, which necessitates two CNOT gate operations. To simplify the representation, these two CNOT gates and the single rotation gate are collectively expressed as an R_{ZZ} gate. In Fig. 5e, the circuit includes edges with path lengths of 1, 2, and 3. For edges with path lengths of 1 and 2, the operations can be efficiently executed using single rotation gates and R_{ZZ} gates. However, for edges with a path length of 3, an R_{ZZZ} gate is applied, which requires two additional CNOT gates compared to the R_{ZZ} gate. Consequently, the circuit in Fig. 5d requires two fewer CNOT gates than the circuit in Fig. 5e.

Although both Fig. 5b, c begin with valid edge assignments, the edge assignment method in Fig. 5c requires additional CNOT gate operations due to differences in the initial assignment compared to Fig. 5b. This suggests that by analyzing the structure

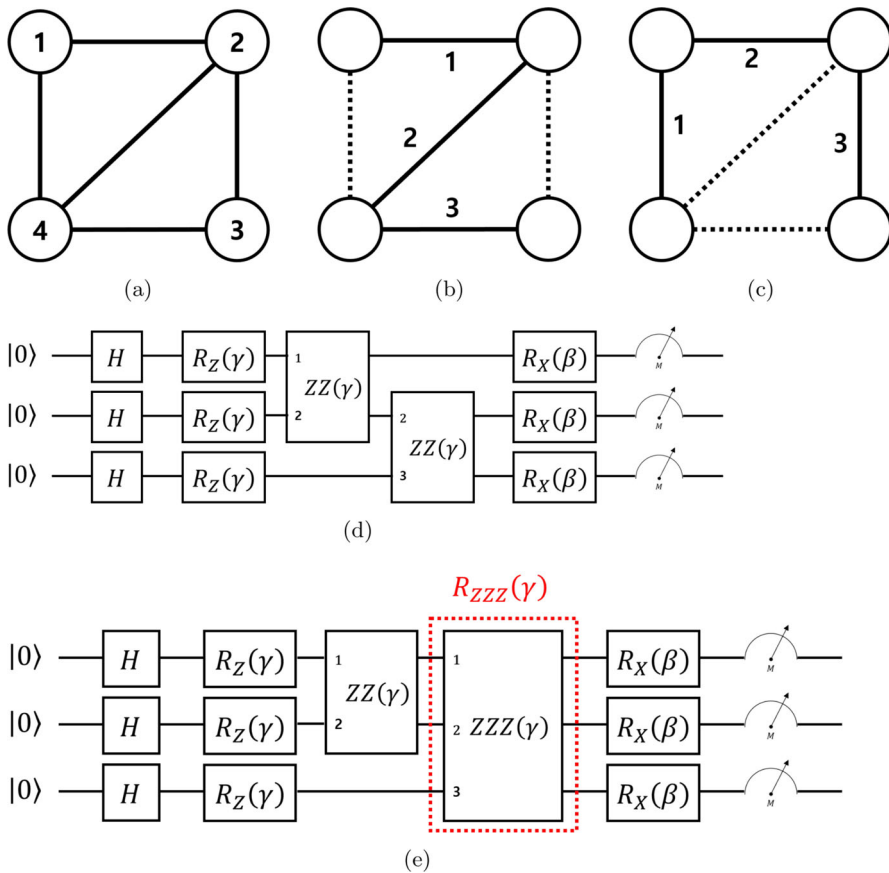


Fig. 5 **a** A square graph with vertices indexed and vertices 2 and 4 connected diagonally. **b** An edge-based graph example for solving the problem, where edges (1, 2), (2, 4), and (3, 4) are indexed as 1, 2, and 3 respectively, and the remaining edges are dashed. **c** Another edge-based graph example for solving the problem, where edges (1, 4), (1, 2), and (2, 3) are indexed as 1, 2, and 3 respectively, and the remaining edges are dashed. **d** Quantum circuit for the edge-based QAOA corresponding to the graph in **b**. **e** Quantum circuit for the edge-based QAOA corresponding to the graph in **c**. The red dotted box represents the rotation operator $R_{ZZZ}(\gamma)$

of the graph and performing more efficient initial edge assignments, the number of CNOT gate operations can be minimized.

Equation (8) takes different forms depending on the edge-assigned graph G_e , and the number of CNOT gates varies according to the path length l . As a result, the number of required CNOT gate operations for the problem Hamiltonian $H_{P'}$ can be expressed as follows:

$$N(H_{P'}) = \sum_{(j,k) \in E} 2(l_{jk} - 1), \quad (16)$$

where $H_{P'}$ represents the problem Hamiltonian constructed based on graphs G and G_e , and l_{jk} denotes the length of the path between vertices j and k in the spanning

tree G_e . Since G_e consists of $n - 1$ edges, the number of edges with $l_{jk} = 1$ is at most $n - 1$. If G_e is constructed as a star graph [29], then the path length l_{jk} for any edge $(j, k) \in E$ is at most 2, resulting in the minimal number of required CNOT gate operations.

For a star graph G_e , the number of required CNOT gate operations for the problem Hamiltonian $H_{P'}^*$ is bounded as follows:

$$2(m - n + 1) \leq N(H_{P'}^*) \leq 2(m - 1). \quad (17)$$

The lower bound is obtained when G_e is a star graph and the number of edges $(j, k) \in E$ with path length $l_{jk} = 1$ is maximized, i.e., $n - 1$ such edges exist. In this case, the remaining $m - (n - 1)$ edges satisfy $l_{jk} = 2$, resulting in a total of $2(m - n + 1)$ required CNOT gates.

The upper bound corresponds to the scenario where only one edge in E has path length $l_{jk} = 1$, and the remaining $m - 1$ edges have $l_{jk} = 2$, leading to $2(m - 1)$ required CNOT gates.

For a graph with m edges, applying vertex-based QAOA requires m R_{ZZ} gate operations, which translates to a total of $2m$ CNOT gate operations. Therefore, adopting a star graph structure for the edge-assigned graph G_e in the edge-based QAOA enables a reduction of at least 2 and at most $2(n - 1)$ CNOT gate operations per iteration compared to vertex-based QAOA.

When the initial edge-assigned graph is not configured as a star graph, the number of required CNOT gates may increase significantly depending on the structure of the assignment. In specific configurations, the edge-based QAOA may even require more CNOT gates than the vertex-based QAOA.

If the information about edge assignments can be tracked, the required number of CNOT gate operations can be reduced. In the edge-based QAOA, the CNOT gate between two qubits serves to modify the edge assignment, which enables dynamic manipulation of edge information during the algorithm. Based on this property, the dynamic edge reassignment (DER) technique can be incorporated into the edge-based QAOA. DER operates by selectively reassigning edge configurations through CNOT gates, allowing the circuit to reduce the total number of required CNOT operations.

The general procedure of DER is as follows. For each edge whose problem Hamiltonian term has $l_{jk} \geq 2$, DER applies additional CNOT gates immediately after the corresponding rotation $\otimes Z$ operation. This additional CNOT operation intentionally modifies the edge assignment graph G_e , which in turn alters the values of l_{jk} for the remaining edges in the problem Hamiltonian. Using these updated values, the algorithm proceeds to the next edge and repeats the same process: apply the problem Hamiltonian operation and strategically insert CNOT gates if applicable. By iteratively performing the problem Hamiltonian operation and the reassignment process, DER enables successive reductions in CNOT gate usage while maintaining logical equivalence.

To examine the specific changes in edge assignment, let us consider the graph depicted in Fig. 2b. Assuming that the edge assignments are as shown in Fig. 2b, we apply a CNOT gate operation where qubit 1 acts as the control qubit and qubit 2 acts as the target qubit.

According to Table 1, the edge-based state $|10\rangle$ corresponds to the vertex-based states $|011\rangle$ and $|100\rangle$. In other words, the $|10\rangle$ state represents the same edge-cut configuration depicted in Fig. 3c. Conversely, the $|11\rangle$ state maps to the vertex-based states $|010\rangle$ and $|101\rangle$. As a result of the CNOT gate operation, the edge-based states $|11\rangle$ and $|10\rangle$ are swapped. However, since the structure of the graph remains unchanged, this transformation can be interpreted as a reassignment of edge allocations rather than a modification of the graph itself. This is functionally equivalent to assigning edge (1, 2) to qubit 1 and edge (1, 3) to qubit 2.

To provide a more intuitive understanding, we examine the quantum state formulation. The quantum state corresponding to Fig. 2b can be expressed as follows:

$$|\rho\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle, \quad (18)$$

where a, b, c , and d are the quantum state coefficients. Applying a CNOT gate operation to Eq. (18) yields:

$$|\rho'\rangle = a|00\rangle + b|01\rangle + c|11\rangle + d|10\rangle. \quad (19)$$

This transformation corresponds to the transition of the edge-assigned graph from Fig. 6a, b. Comparing the states $|10\rangle$ and $|11\rangle$ in $|\rho\rangle$ and $|\rho'\rangle$, both states share the same coefficient c . By analyzing Fig. 6a, b, it can be observed that these states correspond to identical edge-cut configurations. Since this equivalence holds for all coefficients, the CNOT gate only modifies the edge assignment without altering the underlying edge-cut structure.

As a result, the CNOT gate operation modifies the edge assignment. When two connected edges (1, 2) and (2, 3) are assigned to qubits, the CNOT gate operation retains the control qubit's assignment to edge (1, 2) while reassigning the target qubit to the third edge (1, 3), thereby completing the triangle formed by the three vertices. In this process, the newly assigned edge (1, 3) is mapped to qubit index 2.

The CNOT gate in the edge-based QAOA framework plays a critical role in modifying edge assignments. Building on this property, the DER technique strategically introduces additional CNOT operations, through which edge assignments are dynamically altered during circuit construction. By doing so, DER can reduce the overall number of CNOT gates required in the quantum circuit, despite the inclusion of extra gates. To demonstrate how this method works, we examine the graph shown in Fig. 2b and its corresponding quantum circuit depicted in Fig. 4b. Consider the case where an additional CNOT gate operation is applied to the same qubits after executing the R_{ZZ} gate in Fig. 4b. The resulting circuit, with additional CNOT gate, is shown in Fig. 7a.

Since the R_{ZZ} gate consists of two CNOT gates and one single-qubit Z-rotation gate, applying an additional CNOT gate leads to redundancy. Moreover, because the CNOT gate is both Hermitian and unitary, applying two identical CNOT gates results in an identity operation. Thus, the circuit in Fig. 7a is logically equivalent to the circuit in Fig. 7b. Consequently, the edge-assigned graph G_e transitions from Fig. 6a, b.

Since CNOT gates only modify the edge assignment state, if the edge assignments are appropriately tracked, the final quantum state measurements can be mapped to

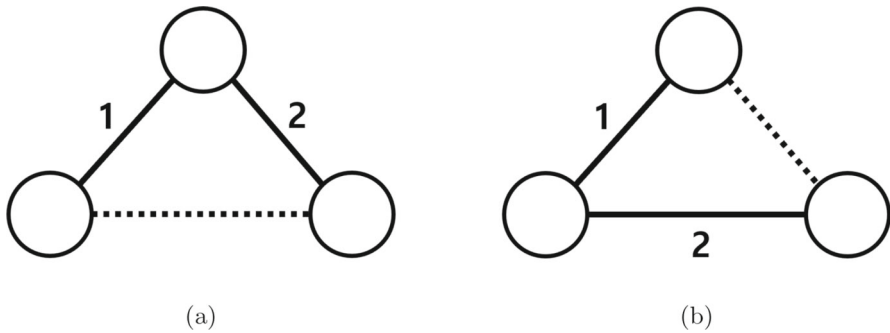


Fig. 6 Example of a triangular graph. **a** Edge-assigned graph constructed based on Fig. 2a. **b** Updated edge-assigned graph obtained by applying a CNOT gate operation between the first and second qubits in Fig. 6a

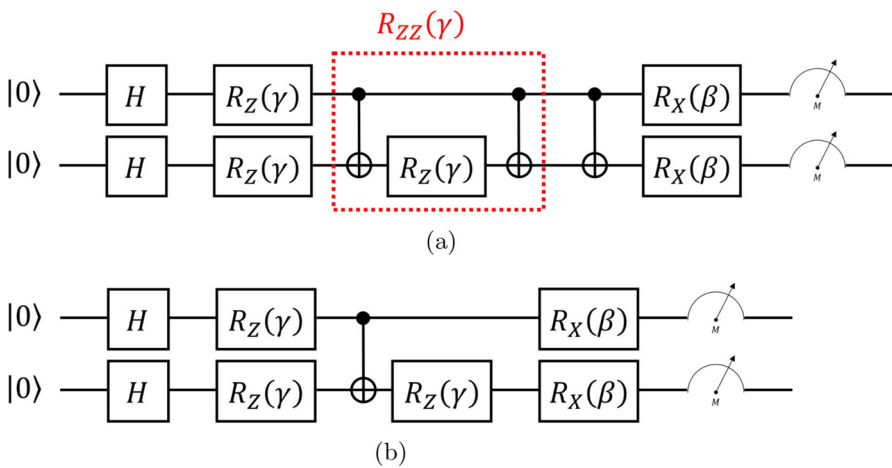


Fig. 7 Quantum circuits for edge-based QAOA with an additional CNOT gate operation. Since the CNOT gate is both a Hermitian and unitary operator, circuits **a** and **b** are logically equivalent despite the difference in the number of gates

vertex states for the vertex-based MAX-CUT problem using the functions $h_{0,1}$ or $h_{1,1}$.

Ultimately, the DER technique applies additional CNOT gates around the rotation $\otimes Z$ gates to construct a logically equivalent circuit that effectively reduces the total number of CNOT gates. However, since CNOT gates dynamically modify the edge-assigned graph, continuous updates to the edge assignment are required. These updates depend on the control and target qubits of the applied CNOT gates, necessitating real-time tracking of the evolving edge-assigned graph structure. A detailed example of the edge-based QAOA operation utilizing DER is provided in Appendix A.

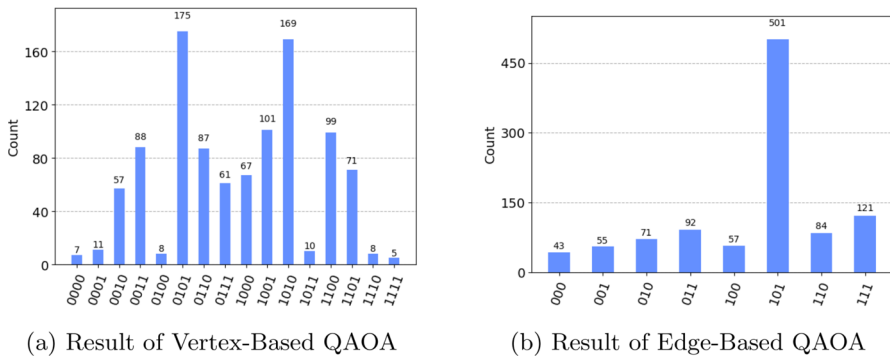


Fig. 8 Simulation results for a simple square graph. The horizontal axis represents the measured quantum states, while the vertical axis indicates the frequency of each quantum state measured out of a total of 1024 measurements. **a** Results for the vertex-based QAOA, where the vertex-based states “0101” and “1010” represent the MAX-CUT states. **b** Results for the edge-based QAOA, where the edge-based state “101” represents the MAX-CUT state

4 Simulation result

In this section, we present simulation results for a simple square graph and various random graphs. All random graphs were generated using the `nx.gnm_random_graph` function from Python’s NetworkX library, and only graphs in which all vertices are connected were selected. This function is well-known for implementing the Erdős–Rényi model [30]. We used IBM Qiskit’s `qasm_simulator` for these simulations. We employed the constrained optimization by linear approximation (COBYLA) optimizer. We omit detailed discussion of the classical optimization problem here.

The simple square graph example is depicted in Fig. 5a. We set the QAOA’s initial angle parameters uniformly to 1.0 for this simulation. Quantum simulation results for the vertex-based QAOA and the edge-based QAOA are illustrated in Fig. 8a, b, respectively. In the vertex-based approach, the states “0101” and “1010” denote MAX-CUT states, while in the edge-based approach, “101” signifies the MAX-CUT state. Although the edge-based method exhibits a higher count for the maximum cut state, the vertex-based method records a lower count for the minimum cut state. Therefore, when comparing the approximation ratios as performance metrics, both approaches achieve similar values of approximately 0.81.

For random graphs with 15 and 20 vertices and 100 or 150 edges, we conducted simulations for three algorithms: vertex-based QAOA, edge-based QAOA, and edge-based QAOA with DER. The results are presented in Fig. 9, where the horizontal axis represents the QAOA level (up to level 3) and the vertical axis indicates the approximation ratio. The red, blue, and green lines correspond to the vertex-based QAOA, edge-based QAOA, and edge-based QAOA with DER, respectively. Solid lines indicate results for graphs with 15 vertices, while dashed lines represent graphs with 20 vertices. Circle markers denote graphs with 100 edges, and triangle markers denote those with 150 edges. For each graph configuration, we randomly initialized the QAOA angle parameters. We simulated each configuration using 10 independently

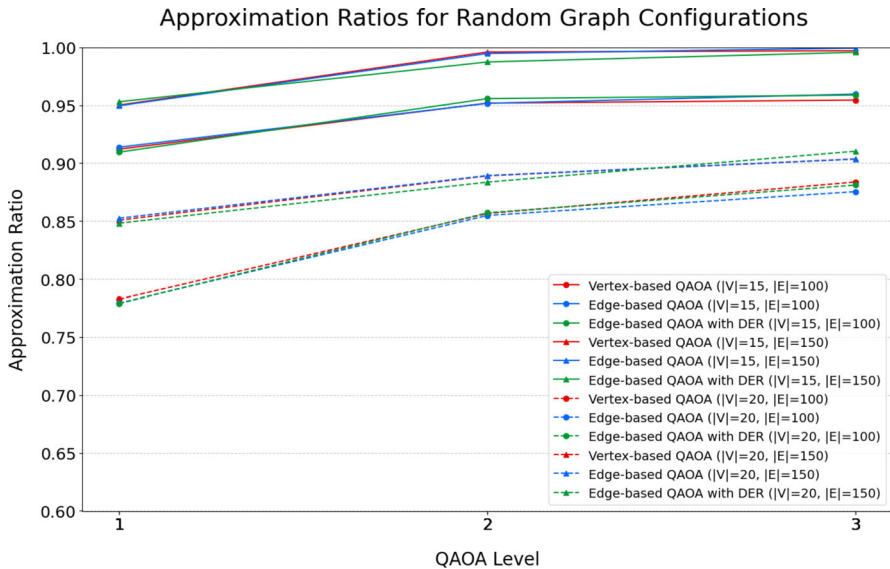


Fig. 9 Approximation Ratios for Different Random Graph Configurations: The figure presents simulation results for vertex-based QAOA, edge-based QAOA, and edge-based QAOA with DER on random graphs with $|V| \in \{15, 20\}$ and $|E| \in \{100, 150\}$. The horizontal axis represents the QAOA level (up to $p = 3$), and the vertical axis indicates the approximation ratio. Red, blue, and green lines correspond to the vertex-based, edge-based, and edge-based with DER methods, respectively. Solid and dashed lines represent graphs with 15 and 20 vertices, respectively, while circle and triangle markers denote graphs with 100 and 150 edges, respectively. The results show that all three algorithms achieve similar approximation ratios, demonstrating that the edge-based QAOA achieves comparable performance to the vertex-based QAOA

generated random graphs, and 100 different initial angle parameter sets were applied across the experiments. Across all QAOA levels, the three algorithms demonstrate nearly identical approximation ratios. This indicates that the edge-based QAOA design can achieve performance equivalent to that of the vertex-based QAOA.

The complexity simulation results for random graphs with 20 vertices are presented in Figs. 10 and 11. We conducted the simulations by fixing the number of vertices to 20 and varying the number of edges as 60, 80, 100, 120, 140, and 160 to generate random graphs. For each case, 1000 random graphs were generated by using different random seeds.

The algorithms employed were vertex-based QAOA, edge-based QAOA, and edge-based QAOA with DER. For both the edge-based QAOA and the edge-based QAOA with DER, the initial edge-assigned graph G_e was constructed as a star graph, centered around the vertex with the maximum degree. In the edge-based QAOA with DER, the logical identity operations that would have resulted from additional CNOT gate applications were not physically executed. Instead of applying the rotation $\otimes Z$ operation along with the additional CNOT gate operations, the circuit was constructed using appropriate CNOT gates and a single rotation Z gate. All three algorithms were run with the same 1000 random seeds for each graph.

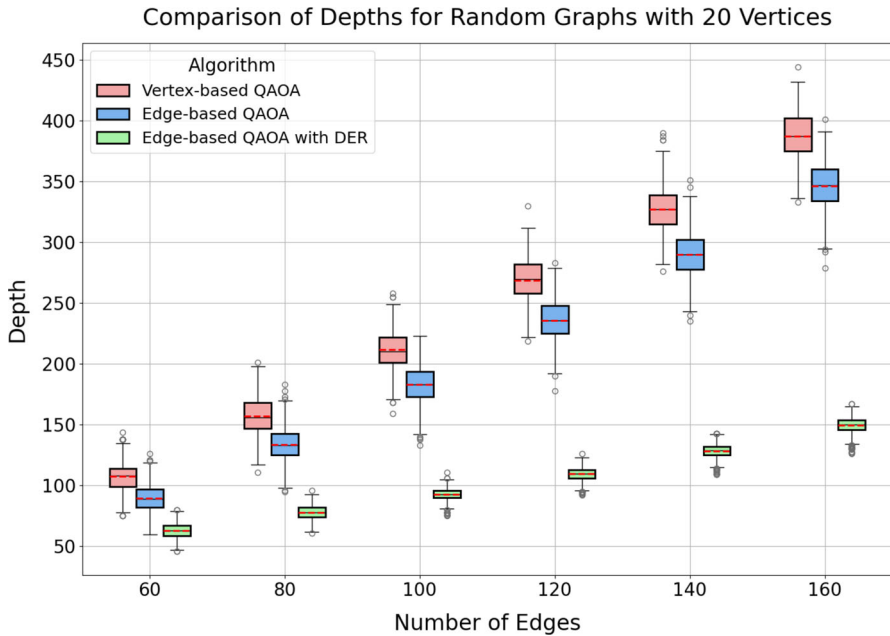


Fig. 10 Comparison of quantum circuit depths for random graphs with 20 vertices across varying edge counts (60, 80, 100, 120, 140, and 160 edges). The vertical axis represents the quantum circuit depth, while the horizontal axis denotes different graph cases. The vertex-based QAOA is represented by red box plots, the edge-based QAOA by blue, and the edge-based QAOA with DER by green. The solid line within each box indicates the median, while the red dashed line represents the mean. The edge-based QAOA demonstrates lower circuit depths than the vertex-based QAOA in all cases. In the case of the edge-based QAOA with DER, lower circuit depths are observed across all cases compared to the vertex-based QAOA, with the gap widening as the number of edges increases

In Fig. 10, the quantum circuit depth was measured using Qiskit's `depth` function. In Fig. 11, the number of CNOT gates was counted after constructing the quantum circuits and applying the `decompose` function in Qiskit. The `decompose` function was applied because the vertex-based QAOA was constructed using R_{ZZ} gates, which were decomposed into two CNOT gates and one single rotation Z gate for precise CNOT gate counting.

Figure 10 shows the comparison of quantum circuit depths. The vertical axis represents the depth of the quantum circuits, while the horizontal axis distinguishes different edge cases with 20 vertices fixed. All three algorithms are visualized using box plots. The vertex-based QAOA is represented with a red background, the edge-based QAOA with a blue background, and the edge-based QAOA with DER with a green background. The median of each box plot is shown with a solid line, and the mean value is indicated by a red dashed line.

The results clearly demonstrate that the edge-based QAOA consistently constructs quantum circuits with lower depths than vertex-based QAOA. Moreover, the depth difference between the two algorithms increases as the number of edges grows. The reason for this is that as the number of edges grows, the degree of the maximum-degree

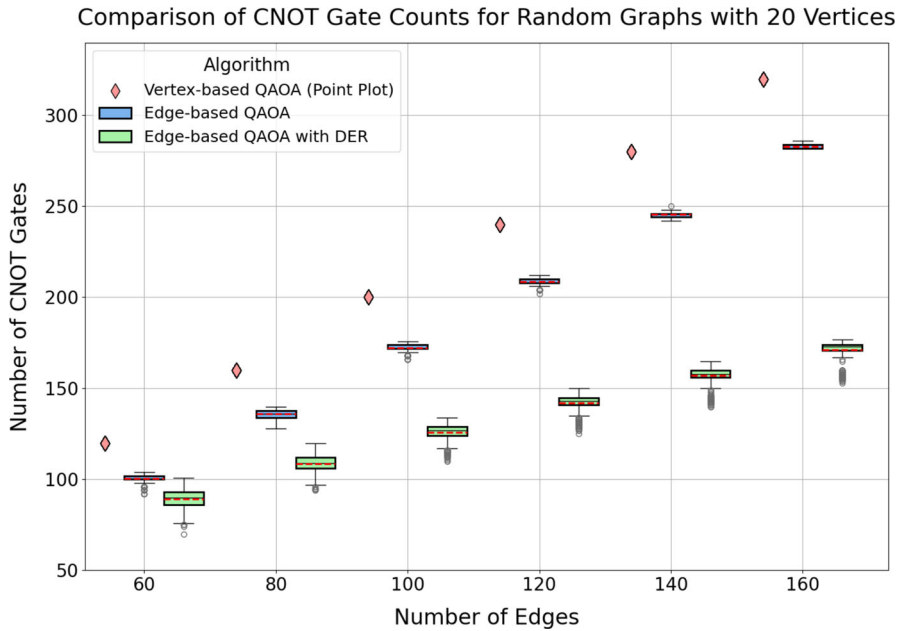


Fig. 11 Comparison of CNOT gate counts for quantum circuits constructed from random graphs with 20 vertices across varying edge counts. The vertical axis represents the number of CNOT gates, while the horizontal axis denotes different graph cases. The vertex-based QAOA is illustrated as a point plot, due to its constant CNOT gate count, which depends only on the number of edges. The edge-based QAOA and the edge-based QAOA with DER are represented by box plots with blue and green backgrounds, respectively. The results highlight the reduction in CNOT gate counts achieved by the edge-based QAOA, especially when the DER method is applied

vertex also increases. Consequently, when forming the edge-assigned graph, the number of edges with path length $l_{jk} = 1$ inevitably increases, leading to further reductions in circuit depth. Considering the previous result in Fig. 9, where the performance of the vertex-based QAOA and the edge-based QAOA was comparable, it becomes clear that the edge-based QAOA can achieve similar performance while generating circuits with lower depths than the vertex-based QAOA. Furthermore, when the DER method was applied by tracking edge reassignments, the circuit depth was further reduced significantly.

For the graphs with 100 edges, the average circuit depth was measured as 211.66 for vertex-based QAOA, 183.23 for the edge-based QAOA, and 93.01 for the edge-based QAOA with DER. In terms of average circuit depth, the edge-based QAOA exhibited a 13.4% reduction compared to the vertex-based QAOA, while the edge-based QAOA with DER achieved a 56.1% reduction.

Figure 11 presents the comparison of CNOT gate counts in quantum circuits. The vertical axis represents the number of CNOT gates, and the horizontal axis distinguishes different edge cases with 20 vertices fixed. For the vertex-based QAOA, the number of CNOT gates remains constant regardless of the graph structure, as the count is determined solely by the number of edges, resulting in a single value displayed as

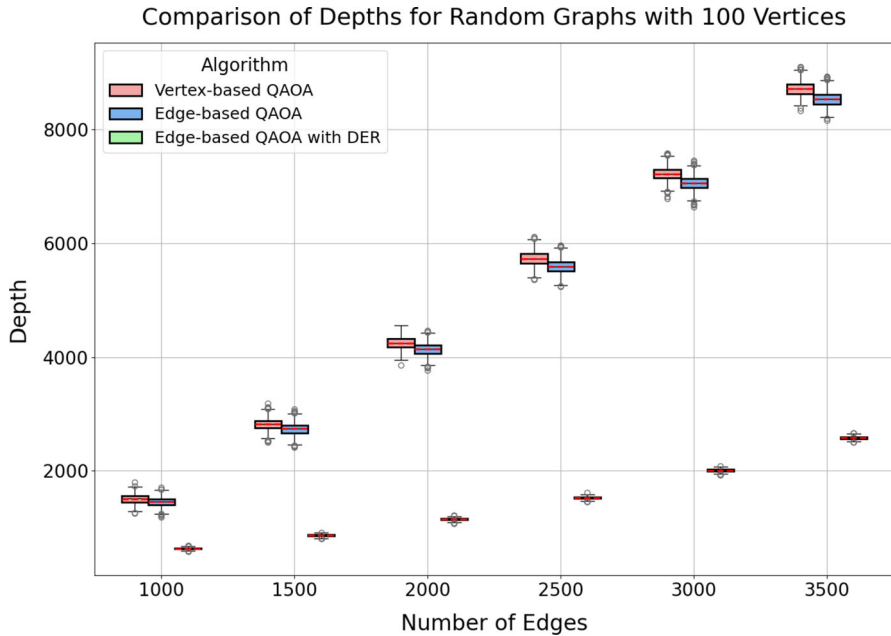


Fig. 12 Comparison of quantum circuit depths for random graphs with 100 vertices across varying edge counts (1000, 1500, 2000, 2500, 3000, and 3500 edges). The graph settings, including the color scheme and statistical indicators, are consistent with those presented for the case with 20 vertices (Fig. 10). The edge-based QAOA consistently achieves lower circuit depths than the vertex-based QAOA, and this difference becomes more pronounced as graph density increases. Furthermore, the edge-based QAOA with DER exhibits significantly lower circuit depths than the vertex-based QAOA across all cases

a point plot. The remaining two algorithms are visualized using box plots, with the same configurations as in Fig. 10.

The results indicate that the edge-based QAOA consistently reduces the number of CNOT gates across all cases. As graph density increases, the number of edges with path length $l_{jk} = 1$ also increases, leading to a further reduction in CNOT gate operations compared to vertex-based QAOA. Furthermore, when applying the DER method, the number of CNOT gates was further minimized, demonstrating its effectiveness in optimizing circuit efficiency.

For the graph with 100 edges, the average CNOT gate count was measured as 200 for vertex-based QAOA, 172.34 for the edge-based QAOA, and 125.85 for the edge-based QAOA with DER. In terms of average CNOT gate count, the edge-based QAOA demonstrated a 13.8% reduction compared to the vertex-based QAOA, while the edge-based QAOA with DER achieved a 37.1% reduction.

Figures 12 and 13 present the comparisons of quantum circuit depths and CNOT gate counts, respectively, for random graphs with 100 vertices. The simulations were conducted by fixing the number of vertices at 100 and varying the number of edges as 1000, 1500, 2000, 2500, 3000, and 3500. A total of 1000 random seeds were used to generate the random graphs for each case, ensuring that the graphs were fully connected.

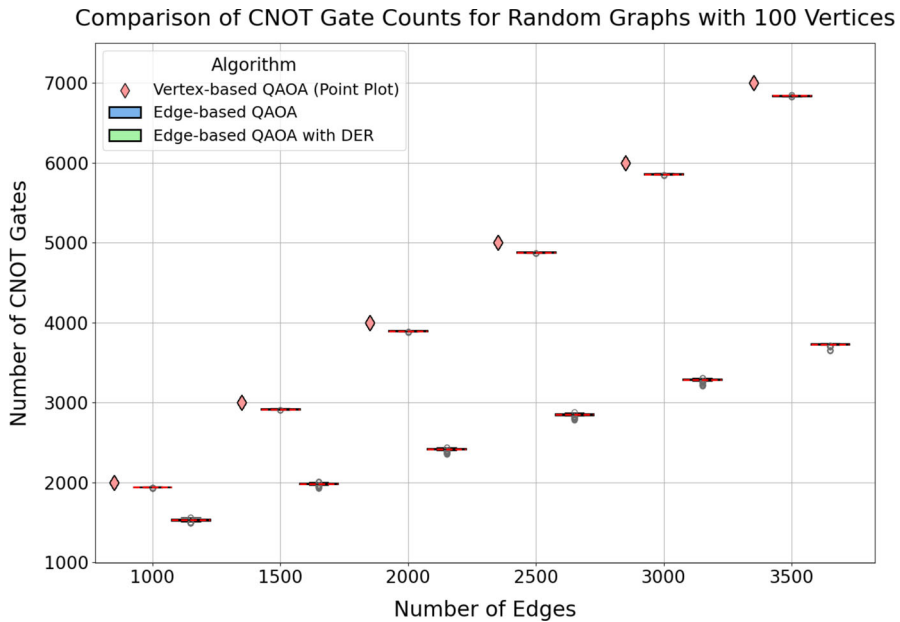


Fig. 13 Comparison of CNOT gate counts for quantum circuits constructed from random graphs with 100 vertices across varying edge counts. The graph settings, including the method of plot and statistical indicators, are consistent with those presented for the case with 20 vertices (Fig. 11). Similar to the circuit depth simulation results, the CNOT gate count for the edge-based QAOA was lower than that of the vertex-based QAOA in all cases. The edge-based QAOA with DER consistently recorded lower CNOT gate counts compared to the vertex-based QAOA

Figure 12 illustrates the comparison of quantum circuit depths. The graph settings, including the horizontal and vertical axes, the types of algorithms, and the graph layout, are consistent with those in Fig. 10. While the results in Fig. 10 focus on simulations with relatively high graph density, the results in Fig. 12 correspond to simulations performed on graphs with lower densities. It was previously noted that for graphs with lower densities, the circuit depth difference between the edge-based QAOA and the vertex-based QAOA becomes less significant. The results in Fig. 12 support this observation, showing that for lower-density graphs, the circuit depths of vertex-based QAOA and the edge-based QAOA are nearly identical. However, as the graph density increases, the depth difference between vertex-based QAOA and the edge-based QAOA becomes more pronounced. In the case of the edge-based QAOA with DER, the circuit depth is significantly lower than that of the vertex-based QAOA across all cases.

Figure 13 presents the comparison of CNOT gate counts used in quantum circuits. The configuration of the graph, including the horizontal and vertical axes, the algorithm types, and the layout, follows the same structure as that in Fig. 11. The simulations were conducted using a star graph as the edge-assigned graph, and the CNOT gate bound for the edge-based QAOA on a star graph is given in Eq. (17). The variation in the number of CNOT gates for the edge-based QAOA depends on the number of

edges with path length $l_{jk} = 1$. As a result, in Fig. 13, the variance in the CNOT gate count for the edge-based QAOA was relatively low, leading to a narrow box plot distribution.

The CNOT gate count for the edge-based QAOA was consistently lower than that of vertex-based QAOA across all cases. Furthermore, in higher-density graphs, where the number of edges with path length $l_{jk} = 1$ is expected to be larger, the difference between the edge-based QAOA and vertex-based QAOA became more pronounced. As observed in previous simulations, in the case of the edge-based QAOA with DER, the CNOT gate count remained significantly lower than that of vertex-based QAOA across all cases.

In Figs. 10, 11, 12, 13, complexity simulations were performed for various random graphs with different vertex counts, edge counts, and densities. Summarizing the simulation results, the edge-based QAOA exhibited lower quantum circuit depths and CNOT gate counts than the vertex-based QAOA for all cases. As graph density increased, the differences in circuit depth and CNOT gate counts between the two algorithms became more pronounced. This trend is likely due to the increase in the number of edges with path length $l_{jk} = 1$. For the edge-based QAOA with DER, the edge reassignment process further reduced quantum circuit depths and CNOT gate counts, consistently outperforming vertex-based QAOA across all graph configurations.

5 Discussion

In this study, the proposed edge-based QAOA demonstrated its capability to reduce quantum circuit complexity compared to traditional vertex-based QAOA, confirming the potential of edge-based approaches in the NISQ era. By assigning qubits to edges instead of vertices, our method effectively reduced redundant gate operations and lowered circuit depth, particularly in high-density graphs.

The simulation results showed that the edge-based QAOA maintained approximation ratios on par with the vertex-based approach, while consistently reducing CNOT gate counts and circuit depths. Notably, as graph density increased, the dynamic edge reassignment (DER) method and star-based initialization proved especially advantageous, further decreasing gate overhead. However, a more rigorous mathematical analysis is needed to clarify how edge reassignment influences circuit depth and gate counts across diverse graph topologies. Such theoretical insights could guide practitioners in selecting optimal edge reassignment schemes tailored to specific problem instances.

Although our performance evaluations were conducted in simulated environments, practical hardware constraints—such as noise and decoherence—remain critical factors. The reduced circuit depth observed in edge-based QAOA may enhance resilience to hardware imperfections, but this hypothesis warrants empirical validation on physical quantum processors.

Recent studies have also explored various modifications to the QAOA ansatz, including techniques that adjust circuit structure or gate sequencing to reduce circuit depth and CNOT gate counts. One notable example is the work presented in [31], which proposes a method to significantly reduce CNOT gate usage in vertex-based

QAOA. Although this optimization was originally developed for vertex-based implementations, it is not inherently limited to them and could potentially be adapted for edge-based QAOA as well. While our study did not explicitly compare or integrate such ansatz-level optimizations, we expect that incorporating these techniques into the edge-based framework may yield further improvements in circuit efficiency.

Future research may extend this framework to other quantum optimization tasks, including MAX- k -CUT or weighted MAX-CUT, to evaluate its generality. Likewise, investigating alternative initial states, improved angle parameter optimization strategies, and hybrid techniques (e.g., combining DER with other circuit-level optimizations) could lead to even more efficient quantum circuits. Such advances would further strengthen the practicality and scalability of edge-based QAOA for complex combinatorial problems.

6 Conclusion

In this study, we investigated an edge-based formulation of the quantum approximate optimization algorithm (QAOA) for the MAX-CUT problem, a classical combinatorial challenge. Traditionally, QAOA has been applied in a vertex-based manner, where each vertex is mapped to a single qubit. By contrast, our approach assigns qubits to edges, thereby offering a more direct representation of the cut structure and enabling reductions in circuit complexity.

Sections 2 and 3 establish the theoretical underpinnings of both vertex-based and edge-based QAOA, and introduced novel strategies—such as dynamic edge reassignment (DER) and star-based initialization—to further optimize circuit depth and CNOT gate usage. In particular, we found that the edge-based formulation offers clear advantages when the initial edge-assigned graph is constructed as a star graph, which simplifies the path construction in the cost Hamiltonian. In Sect. 4, we conducted extensive simulations on various random graphs, demonstrating that the edge-based approach maintains competitive approximation ratios while significantly lowering gate counts, especially in denser graphs. These results confirm that an edge-based perspective can effectively address the overhead associated with R_{ZZ} gates in MAX-CUT circuits.

Our findings highlight the utility of assigning qubits to edges: Not only does it preserve solution quality, but it also offers clear circuit-level advantages, such as reduced CNOT gate counts and simplified circuit structure. In particular, the edge-based QAOA with DER showed substantial reductions in CNOT gate counts and circuit depths. However, this improvement was observed only through simulation, and the optimality of CNOT reduction in terms of theoretical bounds or lower limits remains an open question. Moreover, the DER method introduces additional classical overhead, as it requires dynamic edge reassignment during circuit construction or execution, which inherently depends on classical computational resources. This reliance on classical control may pose limitations in fully quantum or latency-sensitive applications.

Overall, this work underscores the potential of edge-based QAOA for solving the MAX-CUT problem, particularly when combined with star-based initialization. Future research may focus on extending the edge-based framework to other cut-related prob-

lems (e.g., MAX- k -CUT or weighted MAX-CUT), experimentally validating these techniques on physical quantum hardware, and developing more principled strategies for edge reassignment and parameter optimization. By systematically refining edge-based quantum algorithm design and addressing its classical quantum coordination challenges, we anticipate broader impact on real-world optimization problems in the NISQ era and beyond.

Appendix A: Example of edge-based QAOA on a 6-vertex graph

In this section, we consider a graph G with six vertices and nine edges, as depicted in Fig. 14a. To assign edges in this graph, a spanning tree structure G_e can be selected, as shown in Fig. 14b. This selection includes five edges, each assigned to a qubit, covering all vertices in the graph G .

Referring to Fig. 14a, b, the maximum path length in Eq. (8) is determined to be 3. Accordingly, Eq. (8) can be reformulated for this specific case as follows:

$$\begin{aligned} H_{P'} = & \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{12}} Z_{f_q(1,2)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{23}} Z_{f_q(2,3)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{34}} Z_{f_q(3,4)} \right) \\ & + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{35}} Z_{f_q(3,5)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{46}} Z_{f_q(4,6)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{13}} Z_{f_q(1,3)} \right) \\ & + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{14}} Z_{f_q(1,4)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{24}} Z_{f_q(2,4)} \right) + \frac{1}{2} \left(I - \bigotimes_{q=1}^{l_{56}} Z_{f_q(5,6)} \right). \quad (\text{A1}) \end{aligned}$$

Here, $f_q(j, k)$ denotes the qubit index of the edge assigned at the q th step along the path between vertices j and k . Among these, the edges (1, 2), (2, 3), (3, 4), (3, 5), and (4, 6) have path lengths of 1; (1, 3) and (2, 4) have path lengths of 2; and (1, 4) and (5, 6) have path lengths of 3.

To clarify this further, consider Fig. 15, which illustrates both the assigned and unassigned edges, along with vertex labels. Solid lines represent assigned edges, while dotted lines indicate unassigned edges. The labels on the vertices are provided for clarity when referencing unassigned edges. For consistency, we use these vertex labels to name the edges hereafter.

In this example, the edges (1, 2), (2, 3), (3, 4), (3, 5), and (4, 6) are assigned to qubits, whereas the edges (1, 3), (2, 4), (1, 4), and (5, 6) remain unassigned. Among the unassigned edges, (1, 3) and (2, 4) are represented by paths of length 2, while (1, 4) and (5, 6) are represented by paths of length 3.

Considering the quantum state defined by Eq. (9), the 1-level edge-based QAOA state is expressed as:

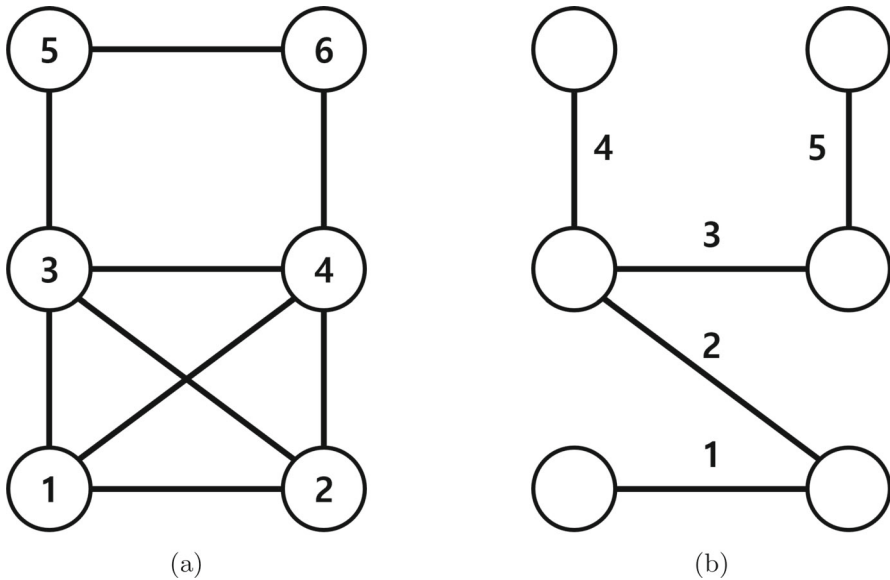
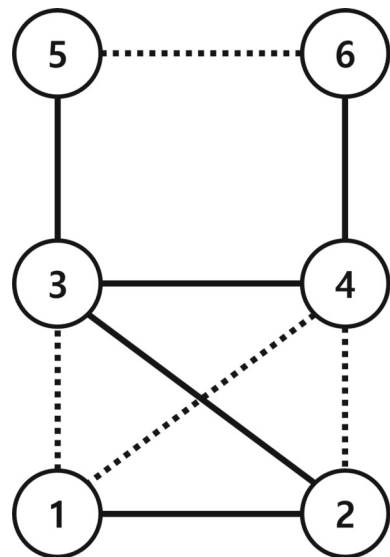


Fig. 14 Illustration of a six-vertex graph and its corresponding spanning tree structure for edge-based QAOA. **a** Original graph with six vertices and nine edges. **b** Selected tree structure containing five edges assigned to qubits, covering all vertices of the graph

Fig. 15 Illustration of the six-vertex graph showing both assigned and unassigned edges for edge-based QAOA. Solid lines represent assigned edges mapped to initial qubits, while dotted lines indicate unassigned edges. Vertex labels are used to reference unassigned edges for convenience



$$|\psi_1(\beta, \gamma)\rangle = e^{-i\beta H_{M'}} e^{-i\gamma H_{P'}} |+\rangle^{\otimes(n-1)} \quad (\text{A2})$$

$$= e^{-i\beta H_{M'}} e^{-i\frac{\gamma}{2} \sum_{(j,k) \in E} \bigotimes_{q=1}^{l_{jk}} Z_{fq(j,k)}} |+\rangle^{\otimes(n-1)}, \quad (\text{A3})$$

where $H_{M'}$ denotes the mixing Hamiltonian of the edge-based QAOA, n is the number of vertices in the graph G , and E represents the edge set of G .

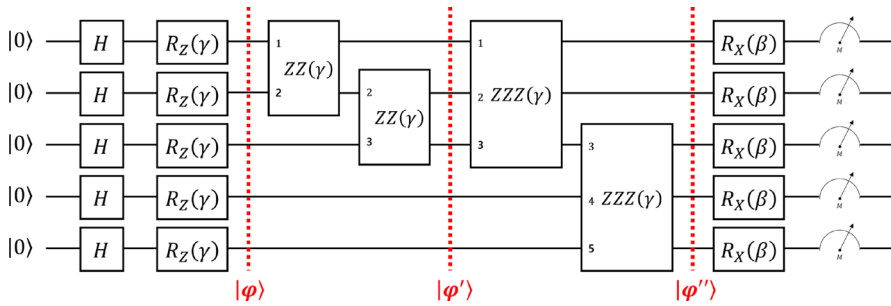


Fig. 16 Quantum circuit representation of the 1-level edge-based QAOA for the six-vertex graph. The state $|\varphi\rangle$ corresponds to the quantum state after applying all single Z-rotation gates for $l = 1$. The state $|\varphi'\rangle$ is obtained after the R_{ZZ} gates for $l = 2$. Finally, the state $|\varphi''\rangle$ represents the quantum state after performing R_{ZZZ} gates for $l = 3$

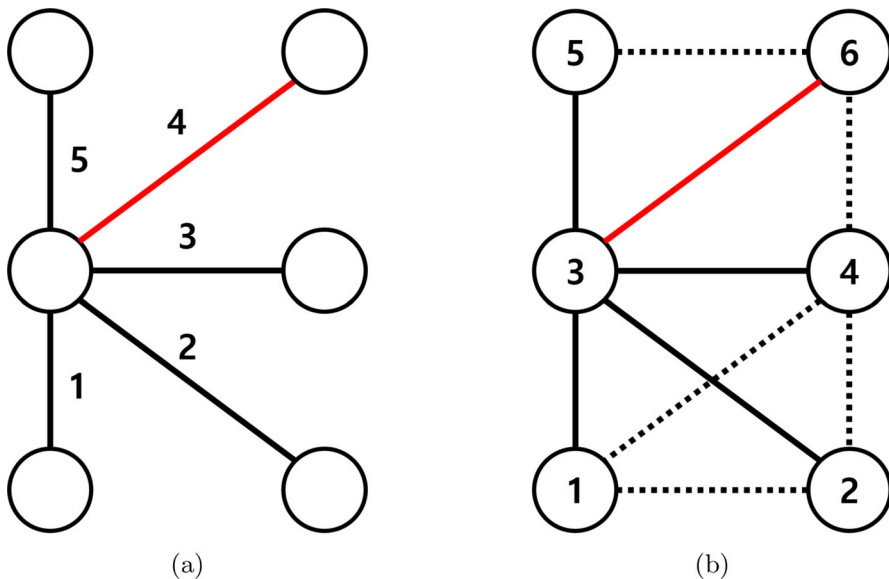


Fig. 17 **a** Edge-assigned graph G_e configured as a star graph. The red line represents an edge assigned to a qubit but not present in the original graph. **b** Same graph with vertex indices for convenient notation, where dotted lines indicate edges that exist in the original graph but are unassigned

In Fig. 16, the quantum state after completing all single Z-rotation operations for $l = 1$ is denoted by $|\varphi\rangle$. For $l = 2$, we apply R_{ZZ} gates to edges (1, 3) and (2, 4), which correspond to qubit pairs (1, 2) and (2, 3) via the function f . Similarly, for $l = 3$, R_{ZZZ} gates are applied between qubits (1, 2, 3) and (3, 4, 5). The quantum states after completing all operations for $l = 2$ and $l = 3$ are denoted by $|\varphi'\rangle$ and $|\varphi''\rangle$, respectively.

Suppose we do not assign the edge-based graph G_e as in Fig. 14b, but instead form a star graph based on the maximum-degree vertex. Let us examine how the results change in this configuration. First, one can configure G_e as shown in Fig. 17a. Here,

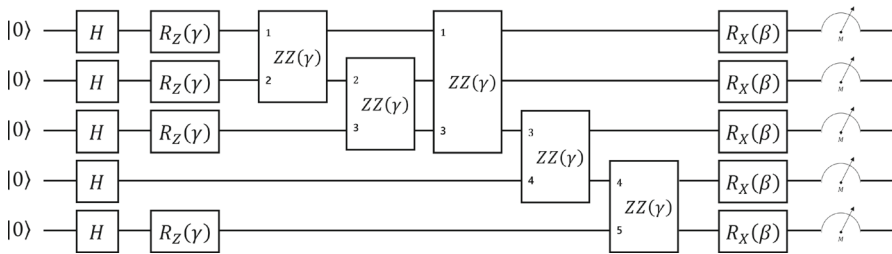


Fig. 18 Quantum circuit of the edge-based QAOA when the edge-assigned graph is configured as a star graph. Because of the star graph's characteristics, all edges have path lengths of either 1 or 2, implying that the circuit can be constructed solely using single Z -rotation and R_{ZZ} gates

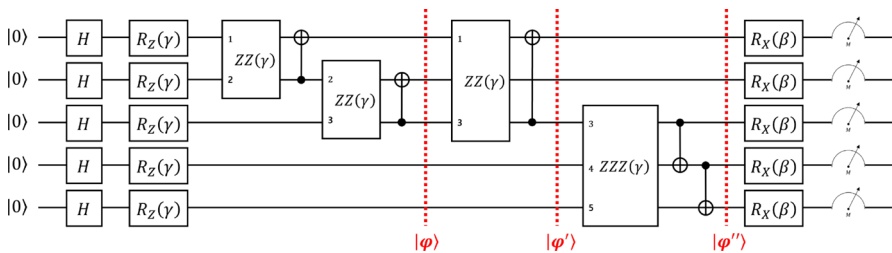


Fig. 19 Circuit representation of the edge-based QAOA with DER for the edge-assigned graph in Fig. 14b. The intermediate quantum states $|\varphi\rangle$, $|\varphi'\rangle$, and $|\varphi''\rangle$ are included to illustrate changes in the edge-assigned graph during circuit execution

the red line indicates an edge assigned to a qubit but absent from the original graph Fig. 14a. For convenience in notation, Fig. 17b shows the same graph with explicit vertex indices. Note that edge (3, 6) does not exist in the original graph Fig. 14a. Because an edge absent from the original graph was used, only four edges have a path length of 1, whereas the previous configuration in Fig. 15 had five such edges. In contrast, five edges now have a path length of 2, and due to the star graph property, no edge has a path of length 3.

Consequently, the circuit in Fig. 18 utilizes four single Z -rotation gates and five R_{ZZ} gates. Recalling that each R_{ZZ} gate decomposes into two CNOT gates and one single rotation gate, whereas each R_{ZZZ} gate involves four CNOT gates and one single rotation gate, we find that the circuit in Fig. 16 uses a total of 12 CNOT gates, while Fig. 18 uses only 10.

By applying the DER method to the graph in Fig. 15, one can obtain the circuit shown in Fig. 19. In Fig. 19, certain R_{ZZ} and CNOT operations combine to form a logical identity, allowing further reduction of gates. After removing these redundant gates, the circuit becomes Fig. 20, using a total of five CNOT gates—seven fewer than in Fig. 16.

If the evolution of the edge-assigned graph can be effectively tracked, the edge-based QAOA with DER can significantly reduce the number of CNOT gates. This evolution is illustrated in Fig. 21, which is derived from the circuit in Fig. 19. In Fig. 21, solid lines represent assigned edges, dotted lines indicate unassigned ones, and the red line highlights an edge that was not included in the original graph Fig.

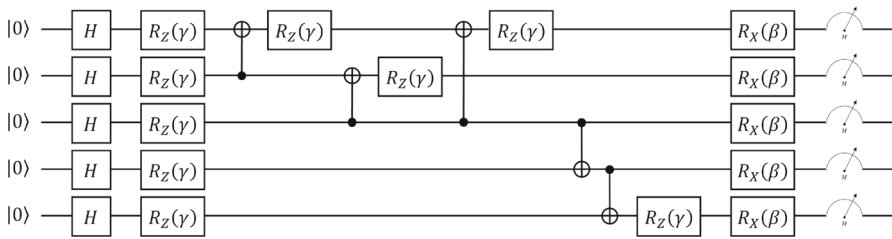


Fig. 20 Practical circuit for operating the edge-based QAOA with DER. After R_{ZZ} and R_{ZZZ} operations, additional CNOT gates are applied to reduce the total number of CNOT gates. One must track how each CNOT gate modifies the edge-assigned state

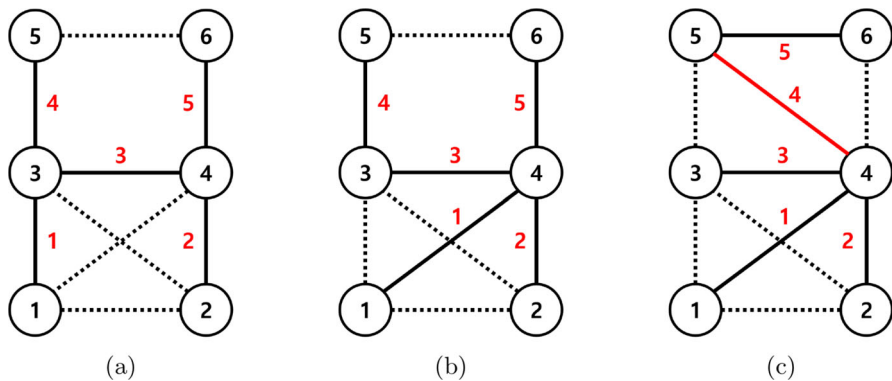


Fig. 21 Evolution of the edge-assigned graph during the edge-based QAOA with DER. **a** corresponds to the quantum state $|\varphi\rangle$, **b** corresponds to $|\varphi'\rangle$, and **c** corresponds to $|\varphi''\rangle$. In **c**, the red line indicates an edge that does not exist in the initial graph Fig. 15. All these edge-assigned graphs maintain a spanning tree structure. In each graph, black numbers denote vertex indices, while red numbers represent the qubit labels corresponding to the assigned edges. For example, in **a**, the red number 4 between vertices 3 and 5 indicates that the edge (3,5) is assigned to the fourth qubit

15. To indicate the qubit assigned to each edge, red numbers are placed next to the corresponding edges. Figure 21a–c shows snapshots of the edge-assigned graph at different stages, all of which preserve a spanning tree structure. Moreover, as long as this structure is maintained, the edge-assigned graph can be flexibly reconfigured—even including edges not present in the original graph.

Acknowledgements This research was supported by Quantum Technology R&D Leading Program (Quantum Computing (RS-2024-00431853) through the National Research Foundation of Korea (NRF) funded by the Korean government (Ministry of Science and ICT(MSIT)). This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2022-0-00463, Development of a quantum repeater in optical fiber networks for quantum internet). This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2023-00225385, Development and application of low-overhead and high-efficiency NISQ-based quantum error mitigation technology).

Author Contributions YJS conceptualized the proposed method, conducted the simulations, derived the equations, and created the figures. YJS and JH collaboratively developed the overall idea of the paper, with JH overseeing the entire project.

Funding Not applicable.

Data Availability No datasets were generated or analyzed during the current study.

Declarations

Ethical approval and consent to participate Not applicable.

Consent for publication The authors agree to publication in the journal. And, the work has not been published or presented elsewhere in part or in entirety.

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Preskill, J.: Quantum computing in the NISQ era and beyond. *Quantum* **2**, 79 (2018). <https://doi.org/10.22331/q-2018-08-06-79>
2. Harrigan, M.P., Sung, K.J., Neeley, M., Satzinger, K.J., Arute, F., Arya, K., Atalaya, J., Bardin, J.C., Barends, R., Boixo, S., et al.: Quantum approximate optimization of non-planar graph problems on a planar superconducting processor. *Nat. Phys.* **17**(3), 332–336 (2021)
3. Wang, S., Fontana, E., Cerezo, M., Sharma, K., Sone, A., Cincio, L., Coles, P.J.: Noise-induced barren plateaus in variational quantum algorithms. *Nat. Commun.* **12**(1), 6961 (2021). <https://doi.org/10.1038/s41467-021-27045-6>
4. Bravo-Prieto, C., LaRose, R., Cerezo, M., Subasi, Y., Cincio, L., Coles, P.J.: Variational quantum linear solver. *Quantum* **7**, 1188 (2023). <https://doi.org/10.22331/q-2023-11-22-1188>
5. Farhi, E., Goldstone, J., Gutmann, S.: A quantum approximate optimization algorithm (2014)
6. Abrams, D.S., Lloyd, S.: Quantum algorithm providing exponential speed increase for finding eigenvalues and eigenvectors. *Phys. Rev. Lett.* **83**(24), 5162–5165 (1999). <https://doi.org/10.1103/PhysRevLett.83.5162>
7. Aspuru-Guzik, A., Dutoi, A.D., Love, P.J., Head-Gordon, M.: Simulated quantum computation of molecular energies. *Science* **309**(5741), 1704–1707 (2005). <https://doi.org/10.1126/science.1113479>
8. Anschuetz, E.R., Olson, J.P., Aspuru-Guzik, A., Cao, Y.: Variational quantum factoring (2018). <https://arxiv.org/abs/1808.08927>
9. Karamlou, A.H., Simon, W.A., Katabarwa, A., Scholten, T.L., Peropadre, B., Cao, Y.: Analyzing the performance of variational quantum factoring on a superconducting quantum processor. *npj Quant. Inf.* **7**(1), 156 (2021). <https://doi.org/10.1038/s41534-021-00478-z>
10. Cruz, D., Fournier, R., Gremion, F., Jeannerot, A., Komagata, K., Tomic, T., Thiesbrummel, J., Chan, C.L., Macris, N., Dupertuis, M.-A., Javezac-Galy, C.: Efficient quantum algorithms for ghz and w states, and implementation on the ibm quantum computer. *Adv. Quant. Technol.* **2**(5–6), 1900015 (2019). <https://doi.org/10.1002/qute.201900015>
11. Kandala, A., Mezzacapo, A., Temme, K., Takita, M., Brink, M., Chow, J.M., Gambetta, J.M.: Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature* **549**, 242–246 (2017). <https://doi.org/10.1038/nature23879>

12. Fujii, K., Mizuta, K., Ueda, H., Mitarai, K., Mizukami, W., Nakagawa, Y.O.: Deep variational quantum eigensolver: a divide-and-conquer method for solving a larger problem with smaller size quantum computers. *PRX Quantum* **3**, 010346 (2022). <https://doi.org/10.1103/PRXQuantum.3.010346>
13. Wang, Z., Hadfield, S., Jiang, Z., Rieffel, E.G.: Quantum approximate optimization algorithm for maxcut: A fermionic view. *Phys. Rev. A* **97**, 022304 (2018). <https://doi.org/10.1103/PhysRevA.97.022304>
14. Zhou, L., Wang, S.-T., Choi, S., Pichler, H., Lukin, M.D.: Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices. *Phys. Rev. X* **10**, 021067 (2020). <https://doi.org/10.1103/PhysRevX.10.021067>
15. Goemans, M.X., Williamson, D.P.: Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. ACM* **42**(6), 1115–1145 (1995). <https://doi.org/10.1145/227683.227684>
16. Bravyi, S., Kliesch, A., Koenig, R., Tang, E.: Obstacles to variational quantum optimization from symmetry protection. *Phys. Rev. Lett.* **125**, 260505 (2020). <https://doi.org/10.1103/PhysRevLett.125.260505>
17. Bae, E., Lee, S.: Recursive qaoa outperforms the original qaoa for the max-cut problem on complete graphs. *Quantum Inf. Process.* **23**(3), 78 (2024). <https://doi.org/10.1007/s11128-024-04286-0>
18. Herrman, R., Lotshaw, P.C., Ostrowski, J., Humble, T.S., Siopsis, G.: Multi-angle quantum approximate optimization algorithm. *Sci. Rep.* **12**, 6781 (2022). <https://doi.org/10.1038/s41598-022-10555-8>
19. Herrman, R., Treffert, L., Ostrowski, J., Lotshaw, P.C., Humble, T.S., Siopsis, G.: Impact of graph structures for qaoa on maxcut. *Quantum Inf. Process.* **20**(9), 289 (2021)
20. Basso, J., Farhi, E., Marwaha, K., Villalonga, B., Zhou, L.: The quantum approximate optimization algorithm at high depth for maxcut on large-girth regular graphs and the sherrington-kirkpatrick model. *arXiv preprint arXiv:2110.14206* (2021)
21. Zhou, Z., Du, Y., Tian, X., Tao, D.: Qaoa-in-qaoa: solving large-scale maxcut problems on small quantum machines. *Phys. Rev. Appl.* **19**(2), 024027 (2023)
22. Lechner, W., Hauke, P., Zoller, P.: A quantum annealing architecture with all-to-all connectivity from local interactions. *Sci. Adv.* **1**(9), 1500838 (2015)
23. Dłaska, C., Ender, K., Mbeng, G.B., Kruckenhauser, A., Lechner, W., Bijnen, R.: Quantum optimization via four-body rydberg gates. *Phys. Rev. Lett.* **128**(12), 120503 (2022)
24. Weidinger, A., Mbeng, G.B., Lechner, W.: Error mitigation for quantum approximate optimization. *Phys. Rev. A* **108**(3), 032408 (2023)
25. Lechner, W.: Quantum approximate optimization with parallelizable gates. *IEEE Trans. Quant. Eng.* **1**, 1–6 (2020)
26. Pelofske, E., Bärttschi, A., Eidenbenz, S.: Quantum annealing vs. qaoa: 127 qubit higher-order ising problems on nisq computers. In: *International conference on high performance computing*, pp. 240–258 (2023)
27. Pelofske, E., Bärttschi, A., Cincio, L., Golden, J., Eidenbenz, S.: Scaling whole-chip qaoa for higher-order ising spin glass models on heavy-hex graphs. *npj Quantum Inf.* **10**(1), 109 (2024)
28. Zeng, Z.-Z., Lü, Z.-P., Yu, X.-G., Wu, Q.-H., Wang, Y., Zhou, Z.: A memetic algorithm based on edge-state learning for max-cut. *Expert Syst. Appl.* **209**, 118077 (2022). <https://doi.org/10.1016/j.eswa.2022.118077>
29. West, D.B., et al.: *Introduction to Graph Theory*, vol. 2. Prentice Hall, Upper Saddle River, NJ (2001)
30. Erdos, P., Rényi, A., et al.: On the evolution of random graphs. *Publ. Math. Inst. Hung. Acad. Sci* **5**(1), 17–60 (1960)
31. Majumdar, R., Madan, D., Bhounik, D., Vinayagamurthy, D., Raghunathan, S., Sur-Kolay, S.: Optimizing Ansatz Design in QAOA for Max-cut (2021). [arxiv:2106.02812](https://arxiv.org/abs/2106.02812)